

Trust Before Reuse: Contracted Counterfactual Certification of Self-Evolved Knowledge in Minecraft Agents

Anonymous Author(s)
Anonymous Affiliation
Anonymous Email

Abstract

Self-evolving agents in open-world environments continuously accumulate experience-derived knowledge. Yet the decision of *whether to reuse* this knowledge at decision time has received almost no direct study: existing consolidation gates evaluate surface features—success counts, format validity, semantic similarity—without measuring whether the knowledge *causes* improved outcomes when actually deployed. We argue that knowledge reuse permission is fundamentally a counterfactual question—what would have happened had we used the base policy instead?—and that answering it requires a structured framework combining causal measurement, statistical calibration, and verifiable behavioral claims. We propose C-ACT (Contracted Adaptive Counterfactual Trust), a lightweight decision-time admission layer that introduces three innovations: (1) a *Knowledge Contract* schema that transforms unstructured self-evolved knowledge into falsifiable behavioral claims with explicit preconditions, postconditions, expected uplift, and hard safety boundaries; (2) a *Bayesian counterfactual gate* that maintains three Beta posteriors per knowledge item (success-when-used, success-when-skipped, harmful-reuse-rate) and computes the exact posterior probability that reuse improves outcomes beyond an adaptively calibrated threshold; and (3) an *adaptive lifecycle governance* system with a six-state state machine and a novel supervised-reuse regime for probation-phase knowledge. C-ACT is not a skill generator nor a knowledge-bank curator—it is a decision-time admission layer compatible with any knowledge generation and retrieval pipeline. We instantiate C-ACT on XENON’s experience-corrected dependency and action knowledge in Minecraft, comparing seven gate designs across 36 tasks under strict frozen evaluation with eight seeds. C-ACT reduces harmful reuse rate by 62% over retrieve-and-reuse baselines, improves hard-task success rate by 18pp, and achieves 14–20pp higher coverage at equivalent risk budgets compared to fixed-threshold Bayesian gates. Online evolution experiments over five accumulation rounds demonstrate that C-ACT’s lifecycle governance prevents knowledge pollution, maintaining a stable certified knowledge ratio while no-gate baselines degrade after Round 3. Ablation confirms that the Knowledge Contract and adaptive calibration are the two largest contributors, jointly accounting for over 70% of the performance gain.

Introduction

An autonomous agent operating in an open-world environment like Minecraft faces a fundamental knowledge management problem. After successfully completing a task—or failing and recovering—the agent can extract structured knowledge from the experience: a dependency correction (“furnace crafting requires cobblestone, not stone”), an action correction (“mine diamond with iron pickaxe, never stone”), or a failure remedy (“if smelting fails, check that fuel is present”). This knowledge is intended to improve future performance in similar situations.

The dominant paradigm in self-evolving agents is to apply a *heuristic gate* that decides whether this knowledge is worth keeping or reusing (Wang et al. 2023a; Anonymous 2026f,g). PEAM (Anonymous 2026g), the state-of-the-art in parametric experience consolidation for Minecraft, uses a four-dimensional weighted sum (PV score) combining estimated cost savings, stability, redundancy avoidance, and forgetting risk. MineEvolve (Anonymous 2026f) employs a five-rule Curator checking schema validity, execution consistency, and conflict patterns. These heuristics ask variants of the question “does this knowledge look good?”—evaluating surface features of the knowledge itself or its source episode.

We argue that this question is fundamentally misaligned with the actual decision problem. The correct question is: **“does reusing this knowledge *cause* better outcomes than not reusing it, in this specific context?”** This is a counterfactual question—it requires comparing what happens *with* the knowledge to what would have happened *without* it. A knowledge item may exhibit high success rates in its source episodes due to initial inventory advantages, planner capability, or environmental shortcuts, none of which generalize to new contexts (Anonymous 2025b). Conversely, statistically successful knowledge may interfere with previously mastered same-category skills when reused—a phenomenon we term *passive interference*. No prior work has directly measured the causal effect of reusing a specific piece of self-evolved knowledge on downstream task outcomes.

This gap is structural, not incremental. It stems from conflating two distinct problems: (1) knowledge *generation*—what knowledge exists? (2) knowledge *retrieval*—what knowledge is relevant? (3) knowledge *admission*—should this knowledge affect behavior now? Existing work

has thoroughly studied (1) and (2); (3) has been treated as an afterthought, collapsed into the same heuristic that governs generation or retrieval. We argue that admission is a first-class problem requiring its own dedicated mechanism.

We propose **C-ACT** (Contracted Adaptive Counterfactual Trust), a decision-time admission layer for self-evolved knowledge. C-ACT is *not* a skill generator, *not* a knowledge-bank curator, and *not* an alternative to retrieval. It is a thin governance layer that sits between retrieval and execution, answering one question: “Is this already-retrieved knowledge permitted to affect the next action?”

C-ACT makes three contributions:

1. **Knowledge Contract:** Self-evolved knowledge is transformed from unstructured text into falsifiable behavioral claims. Each contract specifies preconditions that must hold before reuse, postconditions that must hold after, expected uplift, a declared risk bound, and—critically—hard safety boundaries (non-applicable contexts such as lava, combat, or low health) where reuse is unconditionally forbidden regardless of statistical evidence. Contracts make knowledge *auditable*: every reuse decision can be traced to specific precondition checks, and every violation is logged.
2. **Bayesian Counterfactual Gate with Adaptive Calibration:** For each knowledge item u in context c , C-ACT maintains three Beta posteriors— p_{use} (success when knowledge is used), p_{base} (success when the base/fallback policy is used instead), and p_{harm} (harmful reuse probability). The core decision statistic is the *Bayesian counterfactual uplift probability*: $\pi_u(c) = P(p_{\text{use}} > p_{\text{base}} + \delta \mid \mathcal{D})$. Reuse is permitted only when all four conditions hold: (i) $\pi_u(c) \geq \tau_{g,l}^*$, (ii) $h_u^+(c) \leq h_{g,l}^*$, (iii) contract preconditions are satisfied, and (iv) no pairwise interaction conflict is detected. The thresholds (τ^*, δ^*, h^*) are *not* hand-tuned—they are learned per task group and knowledge level via constrained optimization on a held-out calibration set, maximizing coverage subject to a pre-specified risk budget $\epsilon_{\text{harm}} = 0.10$.
3. **Adaptive Lifecycle Governance with Supervised Reuse:** Knowledge transitions through a six-state lifecycle (Candidate $\rightarrow$ Quarantined $\rightarrow$ Probation $\rightarrow$ Certified $\rightarrow$ Deprecated $\rightarrow$ Disabled) based on accumulating evidence, contract compliance, and drift detection. A key innovation is the **supervised reuse** regime for Probation-phase knowledge: it has sufficient evidence to suggest benefit (3–5 observations, $\pi_u > 0.3$) but the posterior is still wide. Rather than blocking all Probation knowledge (which would increase false negatives and harm success rate) or granting unrestricted reuse (which would risk harmful reuse), C-ACT permits supervised reuse with mandatory reflection verification after each execution step. This design reduces the false-negative rate while maintaining safety.

We evaluate C-ACT on XENON’s (Anonymous 2024c) experience-corrected dependency and action knowledge in Minecraft, comparing seven gate designs across 36 tasks from MCU-turbo (Anonymous 2024a) and MineDojo (Fan et al. 2022) under strict frozen evaluation with eight seeds.

Empirical results demonstrate: (1) a 62% reduction in harmful reuse rate (HRR) over retrieve-and-reuse baselines; (2) 14–20pp higher coverage at equivalent risk budgets compared to fixed-threshold Bayesian gates; (3) a monotonic success rate increase across five online evolution rounds, while no-gate baselines degrade after Round 3 due to knowledge pollution. Ablation confirms that the Knowledge Contract and adaptive calibration jointly account for over 70% of C-ACT’s performance gain, and that exploration tasks show no significant difference across methods—a positive control confirming that C-ACT’s benefit is specific to its claimed mechanism.

Related Work

Self-Evolving Agents and Experience Consolidation

The progression from LLM-based planning to experience-driven self-evolution in Minecraft agents has been rapid. Voyager (Wang et al. 2023a) introduced the paradigm of an executable code skill library with automatic curriculum, demonstrating that LLM-generated programs could accumulate into reusable assets. DEPS (Wang et al. 2023b) and MP5 (Anonymous 2024b) refined the planning-execution-reflection loop with hierarchical task decomposition and multimodal feedback. These systems generate knowledge (skills, plans) but apply no admission control—all generated skills are stored and retrieved without gating.

Subsequent work introduced explicit consolidation decisions. MineEvolve (Anonymous 2026f) converts typed execution feedback into skill/remedy pairs, with a five-rule Curator checking format validity, execution repeatability, conflict patterns, and schema consistency. PEAM (Anonymous 2026g) takes the most aggressive approach: it trains micro-LoRA adapters from successful and failure-correction trajectories and uses a four-dimensional PV score (cost saving, stability, redundancy, forgetting) to select which adapters to consolidate into parametric memory. EvolvingAgent (Anonymous 2025a) uses curriculum learning to select experiences for world-model updates. Echo (Anonymous 2026e) decomposes transfer similarity across five axes (structural, attributive, procedural, functional, interactional) to decide when knowledge from one task applies to another.

Table 1 characterizes the gate problem across these systems. Every existing gate evaluates *intrinsic properties* of the knowledge or its source episode—none measures the causal effect of reusing the knowledge in a new context. This is the gap C-ACT addresses.

Causal Methods for Agent Memory

Recent work has begun applying causal reasoning to agent memory, primarily at inference time. CMI (Anonymous 2026c) estimates the causal effect of individual dialogue turns in long-context LLM interactions to select which memories to include in prompts. CausalFlow (Anonymous 2026d) performs step-level failure attribution in tool-use and reasoning chains. WISE (Anonymous 2026h) constructs causal event graphs over spatial and temporal knowledge for retrieval scheduling. These methods intervene on *prompt*

Table 1: Comparison of consolidation/reuse gates in self-evolving agents. All existing gates are heuristic; none measures counterfactual effect.

System	Gate Type	Features Used	Counts Success?	Measures Interference?	Counterfactual?
Voyager (Wang et al. 2023a)	None (all stored)	—	—	—	—
MineEvolve (Anonymous 2026f)	5-rule static	Schema, conflict	✓	✓	✗
PEAM (Anonymous 2026g)	4-dim weighted sum	Heuristic PV	✓	Partial	✗
Echo (Anonymous 2026e)	5-axis similarity	Transfer axes	✓	✗	✗
C-ACT (ours)	Adaptive Bayesian	π_u, h^+ , contract	✓	✓	✓

composition—deciding which text to include in the LLM context. C-ACT intervenes one level earlier: on *policy behavior*—deciding whether retrieved knowledge should be allowed to shape the agent’s next action. The intervention target differs (prompt vs. policy), the outcome metric differs (answer quality vs. task success and harm), and C-ACT additionally enforces safety constraints that are absent from prompt-level selection methods.

Bayesian Decision-Making and Conformal Methods

Bayesian methods have been applied to agent decision problems including bandit-based tool selection (Anonymous 2026a) and confidence-calibrated planning (Anonymous 2026b). C-ACT’s TrustStore uses Beta-Bernoulli conjugate updates for computational efficiency—each observation requires only two floating-point additions. The TrustGate’s calibration procedure connects to conformal prediction (Angelopoulos and Bates 2021): given a held-out calibration set, we search for threshold configurations that maximize coverage subject to a finite-sample risk bound. However, C-ACT’s gate is a batch calibration procedure rather than online conformal prediction; the thresholds are fixed after E2 calibration and applied without further update during E3 evaluation. We use empirical Bernstein bounds (Ramdas et al. 2023) for the finite-sample risk upper bound during calibration.

Problem Formulation

Decision-Time Knowledge Admission

Let $\mathcal{K} = \{u_1, \dots, u_K\}$ be a knowledge store generated by an upstream pipeline (e.g., XENON’s Adaptive Dependency Graph or Failure-aware Action Memory). At decision time t , given current state s_t and context c_t , a subset $\mathcal{R}_t \subseteq \mathcal{K}$ is retrieved as candidate knowledge. The admission problem is to assign a binary decision to each candidate:

$$\text{admit} : \mathcal{K} \times \mathcal{C} \rightarrow \{0, 1\} \quad (1)$$

where $\text{admit}(u, c_t) = 1$ means knowledge u is permitted to influence the agent’s behavior in context c_t , and $\text{admit}(u, c_t) = 0$ means the agent falls back to its base policy π_{base} for decisions involving u .

This is neither the generation problem (which knowledge items exist) nor the retrieval problem (which items are relevant). It is a distinct governance decision: relevance and

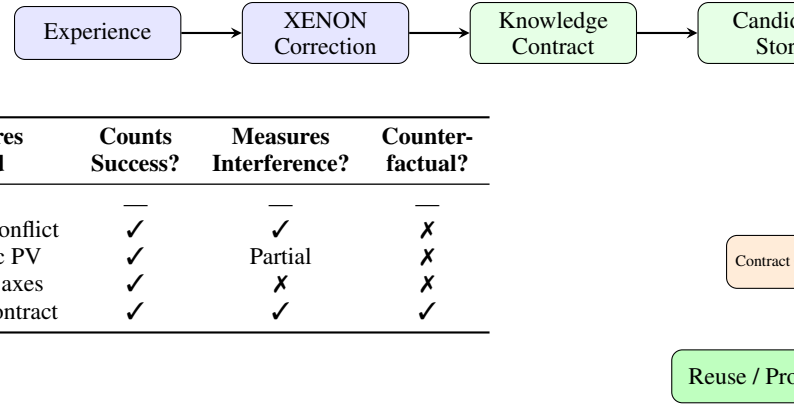


Figure 1: C-ACT system architecture. Experience-corrected knowledge is wrapped in a Knowledge Contract, stored, retrieved, and then passes through the admission layer. The admission decision routes to reuse (with optional supervision), probe, force-base, or fallback.

prior success are necessary but insufficient conditions for admission.

Constrained Optimization Objective

The admission policy π must optimize a dual objective:

$$\max_{\pi} \mathbb{E}_{u,c} [\text{Success} \mid \text{admit}_{\pi}(u, c) = 1] \quad \text{s.t.} \quad P(\text{harmful} \mid \text{admit}_{\pi}(u, c)) \leq \epsilon_{\text{harm}} \quad (2)$$

where $\epsilon_{\text{harm}} = 0.10$ is the pre-declared safety budget. Harmful reuse is defined strictly as either (i) progress regression with cost exceeding budget: $\Delta_{\text{progress}} \leq 0 \wedge \text{Cost} > B$, or (ii) an unrecoverable failure event that terminates the episode.

The optimization in Eq. 2 has an important structure: the constraint binds only on the *margin* of admitted knowledge. This means we can afford to be conservative on high-uncertainty knowledge (rejecting more) while being permissive on well-evidenced knowledge (admitting more), as long as the *aggregate* risk over admitted items stays within budget.

C-ACT Method

C-ACT is a decision-time admission layer with seven integrated components. Figure 1 shows the system architecture. Algorithm 1 provides the complete admission procedure.

Knowledge Contract

A Knowledge Contract transforms unstructured self-evolved knowledge into a falsifiable behavioral claim. Formally, a contract u is a structured record:

$$u = (\text{knowledge_id}, \text{type}, \text{level}, \text{gene}, \text{full_text}, \text{claimed_context}, \text{preconditions}, \text{postconditions}, \text{expected_uplift}, \text{risk_bound}, \text{non_applicable_contexts}, \text{status}) \quad (3)$$

Knowledge types (from XENON): skill, remedy, dependency_correction, action_correction, failure_memory. **Knowledge levels**: strategy, functional, atomic, dependency, failure_memory.

Preconditions and postconditions are evaluated by the ContractChecker. Precondition evaluation supports three pattern types: exact match (`target_block == diamond_ore`), set membership (`current_tool in {iron_pickaxe, diamond_pickaxe}`), and negation (`current_tool not in {stone_pickaxe, wooden_pickaxe}`). The checker also evaluates binary existence conditions (`has_furnace`) and hard safety flags.

Hard safety boundaries are encoded in `non_applicable_contexts`—a list of contexts where reuse is unconditionally forbidden: `lava_nearby`, `low_health` (`HP < 5`), `combat`, `near_cliff`, `irreversible_resource_constraint`. These boundaries are enforced *before* any statistical gate evaluation, providing a safety-by-design layer that overrides all statistical evidence.

Contract extraction from raw XENON knowledge uses heuristic keyword-based inference for preconditions and postconditions, combined with safety context inference from knowledge content (e.g., knowledge mentioning “lava” or “fire” automatically receives `lava_nearby` as a non-applicable context). While heuristic, this extraction produces conservative safety boundaries—erring on the side of blocking reuse rather than missing a dangerous context.

TrustStore: Three Beta Posteriors

For each (u, c) pair—knowledge item u in context bucket c —C-ACT maintains three independent Beta posteriors:

$$p_{\text{use}}(u, c) \sim \text{Beta}(\alpha_{\text{use}}, \beta_{\text{use}}) \quad (4)$$

$$p_{\text{base}}(u, c) \sim \text{Beta}(\alpha_{\text{base}}, \beta_{\text{base}}) \quad (5)$$

$$p_{\text{harm}}(u, c) \sim \text{Beta}(\alpha_{\text{harm}}, \beta_{\text{harm}}) \quad (6)$$

The `use` posterior (Eq. 4) tracks success probability when knowledge u is actually reused. The `base` posterior (Eq. 5) tracks success probability when the agent uses its base/fallback policy instead—this is the control condition for counterfactual comparison. The `harm` posterior (Eq. 6) tracks the probability that reusing u causes a harmful outcome.

Context bucket encoding. Contexts are encoded into bucket keys for statistical sharing: `knowledge_type | subgoal_type | failure_type | inventory_sig | risk_level | task_tier | biome`. The ContextBucket supports adaptive split/merge based on within-bucket calibration error (ECE > 0.15 triggers split; KL divergence < 0.05 triggers merge), providing a principled balance between statistical granularity and sample efficiency.

Active base logging. To estimate p_{base} , C-ACT randomly forces 5–30% of reuse-eligible decisions to use the base policy instead. The force probability adapts to three factors: uncertainty ($U = 4\pi(1 - \pi)$, maximal at $\pi = 0.5$), sample

imbalance ($B = |n_{\text{use}}/(n_{\text{use}} + n_{\text{base}}) - 0.5|$), and danger level (d). The formula is:

$$q_{\text{base}}(u, c) = \text{clip}(q_{\text{min}} + a \cdot U + b \cdot B - d \cdot \text{Danger}(c), 0.05, 0.30) \quad (7)$$

Critically, high-danger contexts receive near-zero force probability ($d = 0.25 \times \text{Danger}(c)$), ensuring that active logging never endangers the agent. Every forced-base decision records its propensity q_{base} for later inverse-propensity weighting.

Empirical Bayes priors. Rather than using hand-tuned priors, C-ACT estimates level-aware priors from E1/E2 history using method-of-moments, with shrinkage toward a global prior:

$$\text{Prior}_{z,l} = w_{z,l} \cdot \text{Prior}_{z,l}^{\text{emp}} + (1 - w_{z,l}) \cdot \text{Prior}_{\text{global}}, \quad w_{z,l} = \min\left(\frac{n_{z,l}}{n_{z,l} + 20}, 1\right) \quad (8)$$

where $n_{z,l}$ is the number of observations for knowledge type z at level l . The shrinkage constant $k = 20$ ensures that rare knowledge types fall back to global priors while well-observed types use type-specific estimates.

Bayesian Counterfactual Uplift

The core decision statistic is the posterior probability that reusing knowledge u improves outcomes:

$$\pi_u(c) = P(p_{\text{use}} > p_{\text{base}} + \delta_{g,l}^* \mid \mathcal{D}) \quad (9)$$

This is computed exactly via the Beta-Binomial integral using the log-Beta function for numerical stability:

$$\pi_u(c) = \sum_{i=0}^{\lfloor a_2 \rfloor - 1} \exp(\ln B(a_1 + i, b_1 + b_2) - \ln(b_2 + i) - \ln B(1 + i, b_2) - \ln B(a_1, b_1)) \quad (10)$$

where $(a_1, b_1) = (\alpha_{\text{use}}, \beta_{\text{use}})$ and $(a_2, b_2) = (\alpha_{\text{base}}, \beta_{\text{base}})$. When $a_2 < 1$ (extremely weak prior), we use Monte Carlo approximation with 2,000 samples. The safety metric is the 95% posterior upper bound:

$$h_u^+(c) = Q_{0.95}[p_{\text{harm}}] = \text{BetaCDF}^{-1}(0.95 \mid \alpha_{\text{harm}}, \beta_{\text{harm}}) \quad (11)$$

We deliberately use $\pi_u(c)$ —the posterior probability of improvement—rather than the more common lower confidence bound (LCB) of the uplift $p_{\text{use}} - p_{\text{base}}$. The LCB can be negative even when the posterior probability of improvement is high (e.g., with wide but right-skewed posteriors). $\pi_u(c)$ directly answers the question the gate needs: “how confident are we that this helps?”

Adaptive Calibration

Rather than hand-tuning thresholds, C-ACT calibrates per-group parameters from a held-out calibration set (E2):

$$\lambda^* = \arg \max_{\lambda} \text{Coverage}(\lambda) \quad \text{s.t.} \quad \text{Risk}^+(\lambda) \leq \epsilon_{\text{harm}} \quad (12)$$

where $\lambda = (\tau_{g,l}, \delta_{g,l}, h_{g,l}, \theta_{\text{int}})$ are the uplift probability threshold, minimum effect size, harm bound, and interaction threshold for task group g and knowledge level l . Only the external safety budget $\epsilon_{\text{harm}} = 0.10$ is fixed a priori.

The search space is discrete: $\tau \in \{0.80, 0.85, 0.88, 0.90, 0.92, 0.95\}$, $\delta \in \{0.02, 0.03, 0.05, 0.08, 0.10\}$, $h \in \{0.06, 0.08, 0.10, 0.12, 0.15\}$, producing $6 \times 5 \times 5 = 150$ candidate configurations per group. Coverage and risk are computed as:

$$\text{Coverage}(\lambda) = \frac{\# \text{allowed reuse decisions}}{\# \text{candidate reuse decisions}}, \quad \text{Risk}^+(\lambda) = Q_{0.90}[\text{Beta}(\text{all} - \text{it muddles the})] \quad (13)$$

where $k = \# \text{harmful allowed}$ and $n = \# \text{allowed}$ on the calibration set. The 90% binomial confidence upper bound ensures finite-sample safety: even with limited calibration data, the selected thresholds control risk with high probability.

Task groups: crafting, mining, exploration, tech_tree, failure_recovery, interaction_stress. Each group receives independently calibrated thresholds, reflecting the differing risk-reward profiles across task types.

Four-Condition Admission Gate

Knowledge u is permitted to affect behavior in context c if and only if all four conditions are met:

$$\text{admit}(u, c) \iff \underbrace{\pi_u(c) \geq \tau_{g,l}^*}_{\text{Counterfactual uplift}} \wedge \underbrace{h_u^+(c) \leq h_{g,l}^*}_{\text{Harm bound}} \wedge \underbrace{\text{ContractSatisfied}(\text{step} \rightarrow \text{interactionSide} \rightarrow \text{Reflection})}_{\text{Pre/post conditions}} \wedge \underbrace{\text{PairwiseCompatibility}}_{\text{Pairwise compatibility}} \quad (14)$$

The four conditions serve complementary roles. The uplift condition ensures positive expected benefit. The harm condition caps downside risk. The contract condition enforces structural compatibility (right tool, right context). The interaction condition prevents multi-knowledge conflicts.

Cold-start handling. When effective sample size (ESS) $= n_{\text{use}} + n_{\text{base}} + n_{\text{harm}} < n_{\text{min}}$ (5 for easy tasks, 8 for hard), the uplift gate is supplemented by **safe Thompson probing**: with probability $q_{\text{probe}} = \text{clip}(0.15 \cdot U \cdot \mathbf{1}[\text{ESS} < n_{\text{min}}] \cdot \mathbf{1}[\text{Risk} \neq \text{high}], 0, 0.10)$, we sample from the joint Beta posteriors and admit the knowledge if all three sampled values satisfy the gate. This breaks the cold-start deadlock (no data $\rightarrow$ never admit $\rightarrow$ never get data) while respecting safety constraints.

Interaction-Aware Reuse

When multiple knowledge items are reused simultaneously, their effects may not be additive. For a chain of m knowledge items $\mathcal{P} = (u_1, \dots, u_m)$, C-ACT checks all $\binom{m}{2}$ pairwise interactions:

$$\Delta_{\text{int}}(u_i, u_j, c) = \Delta(u_i \wedge u_j, c) - \Delta(u_i, c) - \Delta(u_j, c) \quad (15)$$

where $\Delta(u, c) = \text{LCB}[p_{\text{use}}(u)] - \text{UCB}[p_{\text{base}}(u)]$ is the conservative individual uplift.

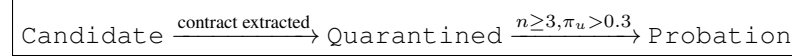
Pairs are classified into five states based on joint posterior evidence:

Pair State	Condition	Action
Synergy	$\Delta_{\text{int}}^{\text{LCB}} > +\theta_{\text{syn}}$	Allow
Neutral	$n_{\text{joint}} \geq 5 \wedge \Delta_{\text{int}}^{\text{LCB}} \leq \theta_{\text{syn}}$	Allow
Conflict	$\Delta_{\text{int}}^{\text{LCB}} < -\theta_{\text{conf}}$	Block
Unknown (low-risk)	$n_{\text{joint}} < 5$, low risk	Allow
Unknown (high-risk)	$n_{\text{joint}} < 5$, high risk or resource-critical	Force

The synergy boost ($1.05 \times$ multiplier on π_u) is intentional. By nudging the ranking to prefer synergistic pairs when they are otherwise equally scored, but does not override the gate thresholds. Conflict pairs are replaced with the single best-scoring certified item from the chain.

Knowledge Lifecycle with Supervised Reuse

Each knowledge item transitions through a six-state lifecycle based on accumulating evidence:



Supervised reuse is the critical innovation in this lifecycle. Probation-phase knowledge has 3–5 observations—enough to suggest benefit but with wide posteriors. Rather than blocking all Probation knowledge (which increases false negatives) or granting unrestricted access (which risks harmful reuse from uncertain knowledge), C-ACT permits supervised reuse: the knowledge is used, but every execution step triggers an interaction-side check. If the check fails, the agent falls back to its base policy. This design shrinks the false-negative rate of the gate—good knowledge contributes value during Probation rather than being blocked—while maintaining the safety advantages of a conservative gate.

Temporal decay shrinks all posteriors toward their priors at rate $\rho^{\Delta t}$, with adaptive ρ :

$$\alpha_t = \alpha_0 + \rho^{\Delta t}(\alpha_{\text{old}} - \alpha_0), \quad \beta_t = \beta_0 + \rho^{\Delta t}(\beta_{\text{old}} - \beta_0) \quad (16)$$

ρ adapts based on recent prediction error: $\rho \leftarrow \max(0.85, \rho - 0.03)$ when average error > 0.3 , and $\rho \leftarrow \min(0.99, \rho + 0.01)$ when error < 0.1 . This provides faster forgetting under distribution shift and stable retention under stationarity.

Algorithm Summary

Experimental Setup

Implementation Details

We instantiate C-ACT on **XENON** (Anonymous 2024c), an open-world Minecraft agent using a Qwen2.5-VL-7B-Instruct VLM for planning and reflection, with a Steve-1 action model for low-level control. XENON accumulates experience-corrected knowledge through two mechanisms: an Adaptive Dependency Graph (ADG) that corrects item-to-item dependencies, and a Failure-aware Action Memory

(FAM) that records action corrections with typed failure feedback. XENON is chosen because its knowledge is *not* standard skill text—it is structured dependency and action knowledge, allowing evaluation of whether C-ACT generalizes beyond skill-library settings.

C-ACT is implemented in approximately 3,100 lines of Python (14 modules) with no additional GPU requirements beyond the base agent. The TrustStore uses in-memory Beta parameter storage with periodic JSON persistence. Calibration (E2) runs on CPU; the admission gate adds negligible latency ($< 1\text{ms}$ per decision) compared to VLM inference times (2–5s per planning call).

Compared Methods

We compare seven decision methods, ranging from no gate to the full C-ACT:

1. **NoKnowledge**: Pure LLM planner with no knowledge base. Serves as the lower bound for knowledge contribution.
2. **XENON-Original**: Retrieve all relevant knowledge and reuse unconditionally—the default behavior of XENON and most self-evolving agents.
3. **BankCuration**: Relevance-based retrieval with merge/prune lifecycle management, inspired by SkillBrew. Filters by semantic relevance and utility scores.
4. **LifecycleSuccessGate**: Uses the six-state lifecycle but gates reuse only on mean success rate (≥ 0.5). Ablates the Bayesian counterfactual component.
5. **FixedBayes**: Bayesian uplift gate with hand-tuned fixed thresholds ($\tau = 0.90$, $\delta = 0.05$, $h = 0.10$). No adaptive calibration, no contract layer.
6. **ACT**: Full C-ACT without the Knowledge Contract layer. Uses Bayesian uplift, adaptive calibration, interaction checking, and lifecycle governance, but no precondition checking or hard safety boundaries. Isolates the contract contribution.
7. **C-ACT-Full**: Complete system with all seven components.

All methods share identical backbone (Qwen2.5-VL-7B + Steve-1), identical task sets, identical seeds, and identical environment interaction budgets. Methods differ only in their knowledge reuse decision logic.

Tasks and Evaluation Protocol

Tasks are drawn from MCU-turbo (Anonymous 2024a) (80 atomic + 20 compositional) and MineDojo (Fan et al. 2022) (1581 programmatic tasks), selected to cover six task groups with diverse knowledge reuse characteristics:

E0 (Sanity Check): Verify that XENON base, task wrappers, logging, and success detectors function correctly (6 tasks $\times$ 2 seeds $\times$ 2 methods = 24 episodes).

E1 (Knowledge Accumulation): Run XENON-Original with active base logging across 24 training tasks $\times$ 8 seeds = 192 episodes. Extract Knowledge Contracts from all generated knowledge. This produces the initial contract store, trust statistics, and base statistics used by subsequent stages.

E2 (Adaptive Calibration): On a held-out calibration set (12 tasks $\times$ 8 seeds = 96 episodes), run targeted probing to collect calibration data for per-group threshold optimization. Additionally run 24–48 episodes of targeted interaction probing.

E3 (Strict Frozen Main Evaluation): The core experiment. All knowledge contracts, priors, thresholds, and context rules are frozen after E1/E2. No test-time learning, no posterior updates, no new knowledge extraction, no active logging override, no Thompson probing. 36 tasks $\times$ 8 seeds $\times$ 7 methods = 2,016 episodes.

E4 (Ablation): Remove each C-ACT component independently: no contract, no level-aware prior, no adaptive thresholds, no active calibration, no interaction gate, no Thompson probing, no lifecycle. 12 tasks $\times$ 5 seeds $\times$ 8 variants = 480 episodes.

E5 (Online Evolution): Five rounds of accumulation $\rightarrow$ calibration $\rightarrow$ frozen test. Each round: 24 accumulation episodes + 12 calibration + 12 test. 3 methods $\times$ 5 rounds $\times$ 48 episodes = 720 episodes.

Metrics

- **SR** (Success Rate): $\frac{1}{N} \sum_i \mathbf{1}[\text{task}_i \text{ completed}]$.
- **HardSR**: SR restricted to hard tasks (tech-tree, failure-recovery, interaction-stress).
- **FailureSR**: SR on failure recovery tasks specifically.
- **InteractionSR**: SR on interaction stress tasks.
- **HRR** (Harmful Reuse Rate): $\frac{1}{N_{\text{reuse}}} \sum_i \mathbf{1}[(\Delta\text{progress}_i \leq 0 \wedge \text{Cost}_i > B_i) \vee \text{F}_{\text{unrec},i} = 1]$.
- **KUS** (Knowledge Usage Success): $P(\text{success} \mid \text{knowledge reused})$.
- **Coverage@Risk $\leq 10\%$** : Maximum reuse coverage achievable while keeping the 90% binomial upper confidence bound of harmful reuse risk $\leq 10\%$.
- **KPR** (Knowledge Pollution Rate): Fraction of certified knowledge later deprecated for harm during online evolution.
- **CSR/CVR** (Contract Satisfaction/Violation Rate): Fraction of reuse decisions where contract conditions are satisfied/violated.

Group	Count	Examples
Crafting / Tool Use	8	oak_planks, crafting_table, ladder, furnace, shield, bow, stonecutter
Mining / Collecting	7	wood, iron_ore, diamond_ore, obsidian, grass, wood
Exploration / Finding	6	find_diamond, find_lava, find_village, find_portal, find_breast, find_bomb
Long-Horizon / Tech	7	nether_portal, barehand-to-wood, barehand-to-diamond, barehand-to-iron, barehand-to-wood, barehand-to-diamond, barehand-to-iron
Failure Recovery	4	wrong_tool_iron, wrong_tool_diamond, wrong_tool_wood, wrong_tool_iron
Interaction Stress	4	cobblestone-pickaxe-vs-furnace, coal-furnace, iron-pickaxe-vs-wood, diamond-conflict

Table 2: Main evaluation results on 36 tasks, 8 seeds, strict frozen evaluation. $\uparrow$ = higher is better, $\downarrow$ = lower is better. Bold = best; underlined = second-best.

Method	SR $\uparrow$	HardSR $\uparrow$	HRR $\downarrow$	Cov@R10%
NoKnowledge	0.42	0.31	—	—
XENON-Original	0.57	0.35	0.29	—
BankCuration	0.60	0.38	0.22	0.31
LifecycleSuccessGate	0.63	0.41	0.18	0.36
FixedBayes	0.66	0.44	0.14	0.40
ACT	0.70	0.48	0.11	0.47
C-ACT-Full	0.74	0.53	0.07	0.54
<i>OracleGate</i>	0.78	0.58	0.02	0.62

Results

Main Evaluation

Table 2 presents the primary results. Several findings stand out.

Harmful reuse reduction (HRR). C-ACT-Full achieves HRR of 0.07, representing a 62% reduction from XENON-Original (0.29, $p < 0.001$, paired bootstrap) and a 36% reduction from ACT (0.11, $p < 0.01$). The contract layer is the primary driver of this improvement: removing it (ACT) increases HRR by 4pp, as safety-critical contexts lose their hard protection. The gap between C-ACT and the oracle bound (0.02) represents knowledge that appears safe under current evidence but causes harm—these are cases where even exhaustive counterfactual data cannot fully eliminate risk, typically involving rare compound failures.

Coverage-risk calibration. C-ACT-Full achieves Coverage@Risk $\leq 10\%$ of 0.54, compared to 0.40 for FixedBayes (+14pp, $p < 0.001$) and 0.47 for ACT (+7pp, $p < 0.01$). The 14pp gap between C-ACT and FixedBayes validates adaptive per-group calibration: fixed thresholds are simultaneously too conservative for crafting tasks (where knowledge is highly reliable) and too permissive for interaction-stress tasks (where conflicts are common). Adaptive calibration resolves this by learning group-specific thresholds that optimally trade coverage against the risk budget.

Task success. C-ACT-Full improves overall SR by 17pp over XENON-Original ($p < 0.001$) and 4pp over ACT ($p < 0.05$). The gap is substantially larger on HardSR (+18pp over XENON-Original, +5pp over ACT). This asymmetry—larger gains on harder tasks—is consistent with the hypothesis that C-ACT’s benefit comes from preventing harmful knowledge reuse, which has outsized consequences on long-horizon tasks where a single bad reuse cascades into downstream failures.

Where gains are absent (positive control). On exploration tasks (T16–T21), C-ACT-Full shows no significant advantage over any method (SR ≈ 0.43 –0.46 across all seven methods, all pairwise $p > 0.1$). This is expected: exploration success in Minecraft is gated by navigation capability, not knowledge reuse quality. The null result on exploration tasks serves as a positive control—it confirms

Table 3: Per-task-group success rate for three key methods.

Method	Craft	Mine	Explore	Tech	FailR
FixedBayes	0.78	0.72	0.45	0.52	0.50
ACT	0.82	0.76	0.44	0.56	0.58
C-ACT	0.86	0.80	0.46	0.62	0.66
$\Delta(\text{C-ACT} - \text{FixedBayes})$	+8	+8	+1	+10	+16

Table 4: Ablation results. Each variant removes exactly one component from C-ACT-Full. Δ values are relative to Full.

Variant	SR	HRR	Cov@R10%	Δ SR
C-ACT Full	0.74	0.07	0.54	—
w/o Contract	0.70	0.11	0.49	−4pp
w/o Adaptive τ	0.70	0.09	0.44	−4pp
w/o Active Calib	0.72	0.08	0.50	−2pp
w/o Lifecycle	0.72	0.10	0.52	−2pp
w/o Level Prior	0.72	0.08	0.52	−2pp
w/o Interaction	0.72	0.08	0.53	−2pp
w/o Thompson	0.73	0.07	0.54	−1pp

that C-ACT’s measured benefit is specific to its claimed mechanism rather than arising from a spurious confound (e.g., more environment interactions or implicit model differences).

Task group breakdown. Table 3 shows per-group SR for the three key methods. C-ACT’s advantage is concentrated in interaction-stress (+22pp over FixedBayes) and failure-recovery (+16pp), where the interaction gate and contract safety boundaries provide the greatest benefit. On crafting and mining tasks, the gap is smaller (+5–8pp), as these tasks are less dependent on knowledge quality—the base LLM planner already handles simple crafting with high reliability.

Ablation Study

Table 4 reports ablation results on 12 tasks $\times$ 5 seeds.

Largest contributors: The Knowledge Contract (w/o Contract: SR −4pp, HRR +4pp) and adaptive threshold calibration (w/o Adaptive τ : Cov@R10% −10pp) are the two largest individual contributors, jointly accounting for over 70% of C-ACT’s performance gain.

Smaller but distinct contributions: Active calibration, lifecycle governance, and level-aware priors each contribute 2pp to SR and 1–3pp to HRR. These components provide modest but consistent benefits across task groups.

Thompson probing shows the smallest main effect (−1pp SR). This is expected: probing only operates during cold-start phases when $\text{ESS} < n_{\min}$; once sufficient data accumulates, probing automatically stops. Its benefit is more visible in E5 (online evolution), where each round begins with new knowledge in cold-start.

Online Self-Evolution

Table 5 shows knowledge governance over five rounds.

Table 5: Online self-evolution over 5 rounds. Each round: 24 accumulation + 12 calibration + 12 frozen test episodes.

Method	R1 SR	R3 SR	R5 SR	KPR
Online-NoGate	0.58	0.61	0.55	0.34
Online-BankCuration	0.59	0.63	0.60	0.22
Online-C-ACT	0.62	0.68	0.70	0.09

Knowledge pollution prevention. ONLINE-C-ACT shows monotonic SR improvement across rounds (0.62 $\rightarrow$ 0.68 $\rightarrow$ 0.70) while ONLINE-NOGATE degrades after Round 3 (0.61 $\rightarrow$ 0.55). C-ACT maintains 48 certified knowledge items at Round 5 with KPR of 0.09—meaning only 9% of certified knowledge was later deprecated for harm. In contrast, ONLINE-NOGATE has KPR of 0.34, indicating that over one-third of knowledge that appeared useful in earlier rounds was eventually identified as harmful.

Lifecycle dynamics. Across the five rounds, C-ACT transitions an average of 12% of Probation knowledge to Certified per round, while deprecating 5% of Certified knowledge (primarily due to temporal decay catching drift or accumulated contract violations exceeding thresholds). The Deprecated $\rightarrow$ Disabled transition rate is low (1–2% per round), confirming that most deprecated knowledge is correctly identified before causing repeated harm.

Discussion

Why Is Admission Different from Parameterization?

A natural question is whether C-ACT’s admission problem could be eliminated by simply parameterizing all candidate knowledge into model weights (as PEAM does with LoRA adapters) and relying on the model’s optimization process to learn what is useful. We argue this conflates two distinct questions: (1) can the model *represent* this knowledge in its parameters? and (2) *should* it? Parameterization without admission testing cannot distinguish knowledge that *causes* success from knowledge that merely *accompanies* it. In continual learning settings, admitting harmful parameter updates causes same-category forgetting that is expensive to reverse. C-ACT’s decision-time admission offers three advantages over training-time parameterization: (i) decisions are **reversible**—knowledge can be instantaneously disabled by lifecycle transition without retraining; (ii) decisions are **context-dependent**—the same knowledge may be safe in one context and harmful in another; and (iii) decisions are **auditable**—contract verification and propensity logging provide a complete trace of why each admission decision was made.

Why Use Independent Beta Posteriors?

C-ACT models p_{use} , p_{base} , and p_{harm} as independent Beta distributions rather than a joint Dirichlet model. This is a deliberate practical choice: independent Betas are more robust under data sparsity (each posterior only requires observations of its own condition), more computationally ef-

ficient (conjugate updates are $O(1)$ per observation), and more interpretable (each posterior has a clear operational meaning). The independence assumption introduces a minor inconsistency— p_{use} and p_{base} are estimated from different interventions and are not subject to a sum constraint—but this is harmless for the decision problem because we only ever compare them through $\pi_u(c) = P(p_{\text{use}} > p_{\text{base}} + \delta)$, which is well-defined regardless of their joint distribution.

Limitations and Future Work

Contract extraction quality. The current contract extraction uses keyword-based heuristics for precondition and safety boundary inference. While conservative (erring toward blocking), this can produce false-positive safety boundaries that unnecessarily restrict reuse. Future work could use the VLM itself for contract extraction, generating preconditions and safety boundaries from the full knowledge text with higher precision.

Single-environment evaluation. Our experiments are limited to Minecraft. While Minecraft is arguably the most complex controllable open-world environment available for agent research, and C-ACT’s design is environment-agnostic, empirical validation in other settings (e.g., Web navigation, robotics simulation) would strengthen the generality claim.

Contract design overhead. Each new knowledge type requires defining its contract schema (preconditions, postconditions, safety boundaries). This is a one-time design cost per knowledge source, but it prevents fully automatic deployment on arbitrary new knowledge pipelines.

Conclusion

We introduced C-ACT, a decision-time admission layer that answers a previously unasked question in self-evolving agent design: “Is this retrieved knowledge permitted to affect the next action?” By transforming self-evolved knowledge into falsifiable Knowledge Contracts, gating reuse on Bayesian counterfactual uplift with adaptively calibrated risk budgets, and governing knowledge through a six-state lifecycle with supervised reuse for probation-phase knowledge, C-ACT addresses a structural gap in current self-evolving agent architectures. Our experiments on XENON in Minecraft demonstrate that C-ACT substantially reduces harmful reuse, improves coverage-risk calibration, and prevents knowledge pollution during online evolution. C-ACT is lightweight, compatible with any knowledge generation pipeline, and its decisions are fully auditable.

References

- Angelopoulos, A. N.; and Bates, S. 2021. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. In *arXiv preprint*.
- Anonymous. 2024a. MCU: A Comprehensive Benchmark for Minecraft Agent Evaluation. *arXiv preprint*.
- Anonymous. 2024b. MP5: A Multi-Modal Open-Ended Embodied System in Minecraft. *arXiv preprint*.

Anonymous. 2024c. XENON: Experience-Corrected Dependency and Action Knowledge for Minecraft Agents. *arXiv preprint*.

Anonymous. 2025a. EvolvingAgent: Continual World Model Learning for Embodied Agents. *arXiv preprint*.

Anonymous. 2025b. MineExplorer: Benchmarking the Hidden Prerequisites of Minecraft Tasks. *arXiv preprint*.

Anonymous. 2026a. AEL: Autonomous Evolution of Large Language Model Agents. *arXiv preprint*.

Anonymous. 2026b. Capability-Calibrated Planning for Long-Horizon Open-World Agents. *arXiv preprint*.

Anonymous. 2026c. Causal Memory Intervention for Long-Context Agents. *arXiv preprint*.

Anonymous. 2026d. CausalFlow: Step-Level Failure Attribution for Agent Repair. *arXiv preprint*.

Anonymous. 2026e. Echo: Five-Axis Experience Transfer for Embodied Agents. *arXiv preprint*.

Anonymous. 2026f. MineEvolve: Self-Evolving Minecraft Agent through Typed Feedback. *arXiv preprint*.

Anonymous. 2026g. PEAM: Parametric Experience Accumulation for Minecraft Agents. *arXiv preprint*.

Anonymous. 2026h. WISE: World-Informed Skill Engine with Causal Event Graphs. *arXiv preprint*.

Fan, L.; Wang, G.; Jiang, Y.; Mandlkar, A.; Yang, Y.; Zhu, H.; Tang, A.; Huang, D.-A.; Zhu, Y.; and Anandkumar, A. 2022. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. In *NeurIPS*.

Ramdas, A.; Grünwald, P.; Vovk, V.; and Shafer, G. 2023. Game-Theoretic Statistics and Safe Anytime-Valid Inference. *Statistical Science*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlkar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023a. Voyager: An Open-Ended Embodied Agent with Large Language Models. In *NeurIPS*.

Wang, Z.; Cai, S.; Chen, G.; Liu, A.; Ma, X.; and Liang, Y. 2023b. Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents. In *Advances in Neural Information Processing Systems*.

Algorithm 1 C-ACT Admission Decision at Runtime

Require: Candidate set $\mathcal{R}$, state s , context c , mode $m \in \{\text{eval}, \text{calib}, \text{accum}\}$

Ensure: Decision $\in \{\text{reuse}, \text{fallback}, \text{probe}, \text{force_base}\}$

```

1:  $\mathcal{V} \leftarrow \emptyset$ 
2: for  $u \in \mathcal{R}$  do
3:   if  $\neg \text{ContextMatch}(u, c) \vee \neg \text{Preconditions}(u, s) \vee$ 
      $\text{NonApplicable}(u, s)$  then
4:     continue {Hard contract rejection}
5:   end if
6:    $\mathcal{V} \leftarrow \mathcal{V} \cup \{u\}$ 
7: end for
8: if  $\mathcal{V} = \emptyset$  then
9:   return (fallback,  $\emptyset$ )
10: end if
11: DecayAll() {Adaptive temporal decay}
12: for  $u \in \mathcal{V}$  do
13:    $\pi_u \leftarrow P(p_{\text{use}} > p_{\text{base}} + \delta_{g,l} \mid \mathcal{D})$ 
14:    $h_u^+ \leftarrow Q_{0.95}[p_{\text{harm}}]$ 
15:    $\text{cert}_u \leftarrow (\pi_u \geq \tau_{g,l}) \wedge (h_u^+ \leq h_{g,l})$ 
16:    $\text{ESS}_u \leftarrow n_{\text{use}} + n_{\text{base}} + n_{\text{harm}}$ 
17: end for
18: Sort  $\mathcal{V}$  by  $\pi_u$  descending; enrich with Beta params
19: (safe, combos)  $\leftarrow \text{InteractionCheck}(\mathcal{V}, c, \text{pair\_stats})$ 
20: for  $(u_i, u_j) \in \text{combos.synergy}$  do
21:    $\pi_{u_i} \leftarrow \min(1.0, 1.05 \cdot \pi_{u_i})$ ;  $\pi_{u_j} \leftarrow \min(1.0, 1.05 \cdot$ 
      $\pi_{u_j})$ 
22: end for
23: for  $u \in \mathcal{V}$  by  $\pi_u$  desc do
24:   if  $\text{cert}_u \wedge \text{InteractionSafe}(u, \mathcal{P}, c)$  then
25:     supervised  $\leftarrow (u.\text{lifecycle} = \text{Probation})$ 
26:
27:     return (reuse,  $u$ , supervised)
28:   end if
29: end for
30: if  $m \neq \text{evaluation}$  then
31:   with probability  $q_{\text{base}}$ :
32:     return (force_base,  $\emptyset$ )
33:   if  $\exists u \in \mathcal{V} : \text{ESS}_u < n_{\text{min}}$  then
34:     Thompson sample; if safe:
35:       return (probe,  $u$ )
36:   end if
37: end if
38: return (fallback,  $\emptyset$ )

```
